

Real Tanpura DSP Algorithm Blueprint

1. The Science of Jivari

The soul of the tanpura lies in the Jivari (the bridge). Unlike standard instruments, the contact point between the string and the bridge shifts dynamically. This is modeled as a non-linear boundary condition where the force follows the Hertzian Contact Law: $F = k * \max(0, \text{displacement})^{1.5}$.

2. Modal Synthesis Engine

Modal synthesis simulates the vibration modes of the string. The displacement is the sum of individual modes q_n . The Jivari effect is achieved by coupling these modes through a non-linear bridge force.

```
// Single String Modal Engine
class JivariString {
public:
    struct Mode {
        double q = 0.0, v = 0.0, freq, damping, phi_bridge;
    };
    JivariString(double f0, double fs, int numModes = 30) : dt(1.0 / fs) {
        for (int n = 1; n <= numModes; ++n) {
            Mode m;
            m.freq = 2.0 * M_PI * f0 * n * (1.0 + (n * n * 0.00001));
            m.damping = 0.5 + (n * 0.1);
            m.phi_bridge = std::sin(n * M_PI * 0.05);
            modes.push_back(m);
        }
    }
    float process() {
        double y_bridge = 0.0;
        for (auto& m : modes) y_bridge += m.q * m.phi_bridge;
        double force_b = (y_bridge < -0.0001) ? 5000000.0 * std::pow(-0.0001 - y_bridge, 1.5) : 0;
        double output = 0.0;
        for (auto& m : modes) {
            double a = (-m.freq * m.freq * m.q) - (m.damping * m.v) + (force_b * m.phi_bridge);
            m.v += a * dt; m.q += m.v * dt;
            output += m.v;
        }
        return (float)output * 0.01f;
    }
private:
    std::vector<Mode> modes;
    double dt;
};
```

Real Tanpura DSP Algorithm Blueprint

3. Miraj-Style Body Resonance

To simulate the large teakwood gourd, a series of IIR Biquad Peak Filters is used. Key frequencies include the Air Volume (110Hz), Wood Resonance (280Hz), and Bridge Shimmer (3.5kHz).

```
// Miraj Body Filter Chain
class MirajBodyResonator {
    // Uses a chain of biquad filters to color the raw string sound.
    // Frequencies: 110Hz (Gourd), 280Hz (Wood), 1.2kHz (Formant), 3.5kHz (Shimmer).
    float process(float input) {
        double out = input;
        for (auto& f : filters) out = processBiquad(out, f);
        return (float)out;
    }
};
```

4. Excitation (The Pluck)

A Raised Cosine pulse (0.5ms to 5ms) simulates the finger contact time. This avoids digital clicks and provides the 'warmth' of a real human touch.

```
// Raised Cosine Excitation
void trigger(double intensity, double durationMs, double position) {
    double phase = (2.0 * M_PI * timer) / durationSamples;
    double force = 0.5 * amplitude * (1.0 - std::cos(phase));
    for (int n = 1; n <= modes.size(); ++n) {
        modes[n-1].v += (force * std::sin(n * M_PI * position)) * dt;
    }
}
```

5. Sample-Accurate Sequencer

The sequencer uses a sample counter rather than a system clock to maintain the meditative 'Sa-Pa-Sa-Sa' rhythm without any drift or jitter.